

Dobot Magician E6 quick guide

The short version: switch the arm on, start the program, and drive it. For the operator on the day.

The arm is position controlled and does not feel you. It will not stop for your hand, the bench, or the phantom.

- Hold **L1** or nothing moves. Release it and the arm stops — that is your first reflex, not the emergency stop.
- Keep the **red emergency stop** on the base within reach. Press to stop, rotate to release.
- The **robot base is the centre** of the workspace: the arm reaches 450 mm in every direction, **front and back**. Nobody puts a hand inside it while it is running.

Robot Dobot Magician E6
Controller PS5 DualSense
Arm address 192.168.5.1
Revision 2026-10-04

Unpacking and packing, the status lights, scanning with the probe and troubleshooting are in the full manual:
github.com/k211/DOBOT-NSL

Contents

- 01 Powering the arm on
- 02 Running and stopping
- 03 Joystick controls

Powering the arm on

Powering the arm on

⚠ **Tap it — do not hold it.** A short press **under 0.5 s** switches the arm **on**. **Holding for 1.5 s is the shut-down gesture.** If you press and hold you will see the light flash and then die when you let go — that is the robot booting and then being told to switch off. Every other button on the robot uses long-press to activate; the power button is the exception.

1. Check the e-stop is **not latched**. If pressed in, rotate to release.
2. Switch on at the wall.
3. **Quick tap** the power button on the base. Finger off immediately.
4. Watch the LED ring: blue fast flash (booting, 30–60 s) → **steady blue**.

Steady blue is the ready state — powered and healthy, waiting for the program. See §2 Status lights (online).

Running and stopping

Everything runs from one terminal on the **Linux laptop**. There is **no timer and no auto-shutdown** — it runs until you stop it.

1. Network

Needed after every laptop reboot. The wired adapter carries a static address on the robot's subnet with **no gateway**, otherwise traffic for the robot leaves over WiFi instead.

```
bash nmcli con up dobot-e6 ping -c3 192.168.5.1 # must reply before going further
```

If the profile is ever missing, recreate it once:

```
bash sudo nmcli con add type ethernet ifname enp59s0 con-name dobot-e6 \ ip4 192.168.5.10/24
ipv4.never-default yes
```

2. Load the environment

```
bash cd "/home/yz22/Documents/code/Dobot arm" source setup_dobot.bash
```

3. Send the arm to its teleop home

Run this **before** launching. It needs the robot's command port, which the running program holds open — so it will not work once the launch is up.

```
bash python3 src/dobot_e6_hw/scripts/go_home.py --speed 5
```

It counts down from three before moving, prints joint angles as it goes, and travels slowly. The click as it starts is the joint brakes releasing — normal whenever a disabled arm is enabled.

4. Launch

```
bash ros2 launch dobot_e6_hw real_hw.launch.py robot_ip:=192.168.5.1
```

What a good startup looks like



Watch for these four lines:

```
[dobot_tcp_node]: RequestControl → 0, {}, RequestControl(); [dobot_tcp_node]: EnableRobot → 0, {}, EnableRobot(); [activate_servo]: Servo started – hold L1 and move the sticks to drive. [haptic_feedback]: Calibration complete – peak_cond=16.5 LOWER=24.8 HARDSTOP=44.7
```

Wait for the slow-flashing green light

Do not touch the controller until the LED ring is flashing green slowly. That is the arm's own confirmation that it is enabled and accepting remote commands — about 15 seconds after launch.

Then keep your hands off for a further **25 seconds**, until `Calibration complete` appears. That window measures how the arm behaves at rest to tune the rumble warning. Move the arm during it and the warning will be mis-tuned — you will get buzzing at rest, or no warning when you need one.

Allow about **40 seconds** from launch to driving. More on the lights → (online)

`RequestControl()` is what claims TCP control of the arm. It runs at **every** startup because the grant does not survive a power cycle. No DobotStudio Pro and no Windows machine is needed for this part.

Stopping

Press **Ctrl+C** in the terminal running the launch and wait for the prompt to come back. The software disables the arm on the way out, so the LED returns to **steady blue**. That is the clean, finished state — there is nothing else to shut down.

Homing while the program is running

Use the controller: hold **L1 + L2 + R2** for one second.

`go_home.py` only works with the launch stopped, so it is for before you start or after an error — not during a demonstration.

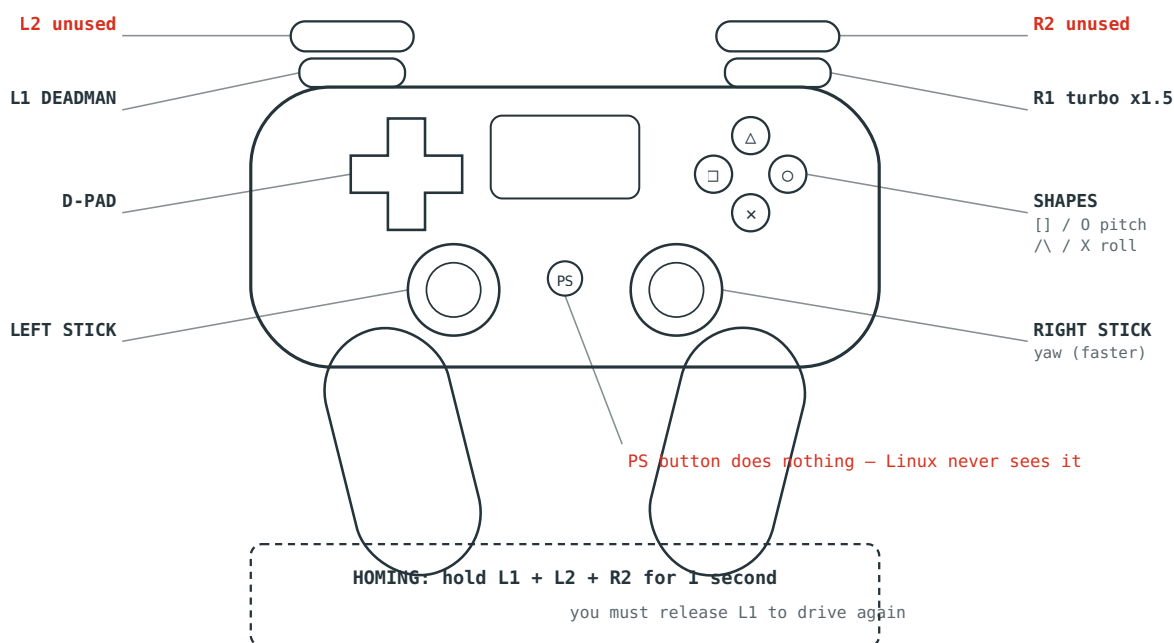
Health checks (optional)

Only needed if something looks wrong. Run in a **second** terminal with the program still going:

```
bash source setup_dobot.bash ros2 topic echo /servo_node/status # 0 = no warnings ros2 topic echo /joint_states --once # do the angles match the real pose? ping -c3 192.168.5.1 # network alive? ls /dev/input/js0 # controller present?
```

Joystick controls

Hold L1 or nothing happens. Every direction is described from the operator's point of view, standing in front of the arm and facing it.



What each control does

CONTROL	MOTION	RATE	FEEL
L1	Deadman — hold it or nothing moves	—	on / off
D-pad ◀ ▶	Move to your left / right	0.03 m/s	full rate or nothing
D-pad ▲ ▼	Move away from / toward you	0.03 m/s	full rate or nothing
Left stick ▲ ▼	Move up / down	0.03 m/s	proportional
Right stick ▲ ▼	Yaw — twist about vertical	0.3 rad/s	proportional
□ / ○	Pitch — tilt the probe nose	0.2 rad/s	full rate or nothing
△ / ×	Roll — rotate about the probe axis	0.2 rad/s	full rate or nothing
R1 (with L1)	Turbo — <i>operator only, see §1 (online)</i>	×1.8	hold
L1 + L2 + R2	Home — return to the safe pose	slow	hold 1 s
L2 or R2 alone	Nothing	—	—
PS button	Nothing — not visible to the software	—	—

The PS button is unusable: the index exists in the controller's button array but the press never reaches the software, because something upstream grabs it. It was deliberately left unused.

Homing

Hold **L1 + L2 + R2 together for one second**. The arm pauses, moves gently to the safe pose, and stops.

Afterwards, teleop stays disabled until you release L1 and press it again. This is deliberate — it stops a stick you are still holding from flinging the arm the instant homing finishes. If the arm seems dead after homing, this is almost always why.

Homing moves joint by joint, so it works even from a pose that ordinary driving cannot escape. But it takes the **direct route with no collision checking** — clear the phantom out of the way and keep a hand near the e-stop.

The rumble

The controller vibrates as the arm approaches a **kinematic singularity**. Stronger buzz means closer. See §7 → (online)

⚠ **The rumble is not contact force.** No force sensor is fitted. The vibration describes the arm's *geometry*, not how hard the probe is pressing. You will feel nothing when the probe touches down.